

1. Scoping using `let` and `const`

Problem 1:

Fix the following code so that `i` is correctly scoped inside the loop and logs 0, 1, 2 after a 1-second delay each.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

Problem 2:

Modify the code to prevent reassignment of `PI` and log its value.

```
let PI = 3.14;  
PI = 3.14159;  
console.log(PI);
```

2. Spread Syntax and Rest Parameters

Problem 1:

Merge two arrays `arr1 = [1, 2, 3]` and `arr2 = [4, 5, 6]` into a single array using the spread operator.

Problem 2:

Write a function `sumAll` that takes any number of arguments and returns their sum using rest parameters.

3. Default Parameters

Problem 1:

Write a function `greet(name, greeting)` where `greeting` defaults to "Hello" if not provided.

Problem 2:

Create a function `createUser(name, age, isAdmin)` where `isAdmin` defaults to `false`.

4. Array/Object Destructuring

Problem 1:

Extract `firstName` and `lastName` from the object:

```
const user = { firstName: "John", lastName: "Doe", age: 30 };
```

Problem 2:

Swap the values of `a` and `b` using destructuring:

```
let a = 5, b = 10;
```

5. Arrow Functions

Problem 1:

Convert the following function to an arrow function:

```
function multiply(a, b) {  
  return a * b;  
}
```

Problem 2:

Fix the `this` issue in the following code using an arrow function:

```
const person = {  
  name: "Alice",  
  greet: function() {  
    setTimeout(function() {  
      console.log(`Hello, ${this.name}!`);  
    }, 1000);  
  }  
};  
person.greet(); // Should log "Hello, Alice!"
```

6. Template Literals

Problem 1:

Rewrite the following string concatenation using template literals:

```
const name = "Bob";  
const age = 25;  
console.log("Name: " + name + ", Age: " + age);
```

Problem 2:

Create a multi-line string using template literals:

```
const title = "My Story";
const content = "Once upon a time...";
// Expected output:
// Title: My Story
// Content: Once upon a time...
```

7. Promises

Problem 1:

Create a `Promise` that resolves after 2 seconds with the message `"Success!"` and log it.

Problem 2:

Simulate an API call that may fail. If successful, resolve with `{ data: "Fetched data" }`. If failed, reject with `"Error fetching data"`.

8. Async/Await

Problem 1:

Rewrite the following `Promise`-based function using `async/await`:

```
function fetchData() {
  return new Promise(resolve => {
    setTimeout(() => resolve("Data received!"), 1000);
  });
}
fetchData().then(data => console.log(data));
```

Problem 2:

Write an `async` function that fetches user data from `https://jsonplaceholder.typicode.com/users/1` and logs the name.

9. Classes and Objects

Problem 1:

Create a `Car` class with properties `brand` and `speed`, and a method `accelerate()` that increases speed by `10`.

Problem 2:

Extend the `Car` class to create an `ElectricCar` class with an additional `battery` property and a `charge()` method.

Node.js and NPM

1. Node.js Introduction

Problem 1:

Write a Node.js script that reads a file `data.txt` and logs its content.

2. Package Managers (npm/yarn)

Problem 1:

Initialize a new Node.js project with `npm init -y` and install the `lodash` package.

Problem 2:

Create a script in `package.json` that runs a JS file using `node`.

3. `npm init` and `npm install`

Problem 1:

Create a new project, install `express`, and save it as a dependency.

Problem 2:

Install `nodemon` as a dev dependency and configure it to run `app.js`.

TypeScript

1. Installing TypeScript

Problem 1:

Install TypeScript globally and verify the version.

2. Compiling TypeScript

Problem 1:

Write a simple TypeScript file (`greet.ts`) that logs "Hello, TypeScript!" and compile it to JS.

3. Setting Up a TypeScript Project

Problem 1:

Initialize a TypeScript project with `tsconfig.json` and configure `"strict": true`.

4. Basic Data Types

Problem 1:

Define variables with explicit types:

- `name` (string)
- `age` (number)
- `isStudent` (boolean)
- `hobbies` (string[])

Problem 2:

Fix the type error:

```
let score: number = "100"; // Error
```

5. Classes in TypeScript

Problem 1:

Create a `Person` class with `name` (string) and `age` (number) properties, and a `greet()` method.

Problem 2:

Add a private property `id` to the `Person` class.

6. Interfaces

Problem 1:

Define an interface `User` with `name` (string) and `email` (string), then create an object that implements it.

Problem 2:

Extend the `User` interface to include an optional `age` property.

7. Inheritance in TypeScript

Problem 1:

Create a `Vehicle` class with `speed` (number) and a `drive()` method. Extend it to create a `Bike` class.

Problem 2:

Override the `drive()` method in the `Bike` class to log "Bike is driving".